Bazzite Unified Home Server + Gaming Station Guide

SSH-first · Zero New Open Ports · Wayland KDE · True Gaming Mode · Headless by Default

SELF-HOSTED CLOUD + GAMING STATION — BAZZITE-DECK EDITION

bazzite-deck · Docker · firewalld · Fail2ban · Nginx Proxy Manager · Nextcloud AIO · DuckDNS · krdp · gamescope-session-plus

A single, self-contained guide that builds a headless home server with two on-demand graphical sessions — KDE Plasma and Gaming Mode — from a single bazzite-deck installation. Supersedes the two-document workflow by integrating every phase in the correct order from the very first boot. No rebase required.

What You Get

Component	What You Get
Headless server default	Boots to an idle SDDM login screen. Docker containers run as systemd services. SSH always on port 53222.
KDE Wayland session	Full KDE Plasma on the physical display, on demand via SSH. Enables krdp remote desktop.
Gaming Mode session	Steam Deck-style gamescope session. Direct GPU scanout, controller-native. Refused if another session is active.
krdp remote desktop	KDE's native Wayland RDP server. Tunneled exclusively through SSH — port 3389 blocked by firewalld.
Nextcloud AIO	Self-hosted cloud: files, calendar, contacts. Behind Nginx Proxy Manager with Let's Encrypt via DuckDNS.
SSH-first design	Every operation is a single SSH command. No new ports beyond 53222 are ever opened.

Architecture Overview

The server runs bazzite-deck — Bazzite's HTPC/living-room image — which ships both gamescope Gaming Mode and KDE Plasma in the same image. On first boot the machine enters Gaming Mode once (expected). We immediately apply a headless SDDM override, so every subsequent boot lands at an idle login screen. Docker containers start at boot via systemd, completely independently of any graphical session.

Action	Flow
Launching KDE	SSH → start-kde.sh → writes one-shot SDDM autologin → restarts SDDM → KDE Wayland appears → headless config restored after session is stable.
Remote desktop	KDE must be running. SSH tunnel forwards port 3389 to localhost. Connect any RDP client to localhost:3389. krdp captures compositor output — monitors can be off.
Gaming Mode	SSH → start-gaming.sh → checks seat0 for active session → writes one-shot SDDM autologin → gamescope session appears. Steam Power menu exits to idle SDDM.

Recommended Minimum Hardware

Component	Minimum / Recommended
CPU	4-core x86-64 minimum · 6-core or better recommended
RAM	8 GB DDR4 minimum · 16 GB+ recommended
OS Drive	120 GB NVMe SSD minimum · 500 GB+ NVMe recommended
Data Drive	Any size HDD or SSD · Dedicated, mounted at /srv/nextcloud-data
GPU	AMD recommended for gaming (plug-and-play) · NVIDIA: beta for Gaming Mode
Network	Gigabit Ethernet · Wi-Fi strongly discouraged for servers

Critical Reading: Common Pitfalls & How This Guide Avoids Them

Pitfall	What Goes Wrong & The Fix
Pitfall 1 — bazzite-deck boots to Gaming Mode on first install	After a fresh install, SDDM autologins into Gaming Mode on the very first boot. Phase 1 turns this into an opportunity: switch to Desktop Mode from within Gaming Mode, enable SSH from the KDE terminal, then apply the headless SDDM override — all in one physical session.
Pitfall 2 — SSH is not enabled by default on Bazzite	Bazzite ships with SSH disabled. The very first action in Phase 1 Step 2 is enabling sshd from the KDE terminal. All subsequent steps are done remotely over SSH.
Pitfall 3 — SDDM reads config once per start; timing matters	The launch scripts write a one-shot autologin config, restart SDDM (which reads it and autologins), then restore the headless config in a background subshell once SDDM is confirmed active plus a 5-second buffer. The subshell redirects stdout/stderr to /dev/null so the SSH terminal returns immediately.
Pitfall 4 — krdp listens on ALL interfaces, not just localhost	krdp listens on 0.0.0.0 (all interfaces). Port 3389 is reachable on your LAN unless blocked. firewalld blocks it because we never open 3389. Phase 9 includes an explicit firewall verification step. Never open port 3389 in firewalld.
Pitfall 5 — krdp requires an active KDE session	krdp shares the running KDE compositor session. If no KDE session is active, krdp is not running and RDP connections will be refused. Always run start-kde.sh first.
Pitfall 6 — Sudoers entries require absolute paths and root-owned scripts	sudo never expands ~ or $HOME in sudoers files. Scripts with NOPASSWD sudo access must be owned by root and stored in a path the regular user cannot write to. This guide uses /usr/local/bin/ with root:root ownership.
Pitfall 7 — SELinux blocks sshd on non-standard ports	Bazzite runs SELinux in enforcing mode. Only port 22 is labelled ssh_port_t by default. Restarting sshd on port 53222 without updating the policy causes sshd to silently fail to bind. Fix: semanage port -a -t ssh_port_t -p tcp 53222 before restarting sshd.
Pitfall 8 — Package installation must precede security hardening	The SSH hardening phase needs semanage and fail2ban — both installed via rpm-ostree. Attempting security hardening before installing these tools causes silent failures. This guide installs all packages in a dedicated phase before any hardening begins.
Pitfall 9 — Auto-suspend silently kills all containers	Bazzite can suspend after 15 minutes of inactivity, silently killing all Docker containers. Phase 1 Step 3 permanently disables all suspend targets — including suspend-then-hibernate — at the OS level before anything else.
Pitfall 10 — Docker bypasses firewalld for published ports	Docker injects nftables rules directly, bypassing firewalld for any port mapped in a Compose file. Only add firewalld rules for SSH (port 53222). Never add rules for ports 80, 443, or 3389.
Pitfall 11 — Missing nofail on secondary drive bricks headless boot	Without nofail, a failed or slow-to-spin-up data drive causes systemd to drop into emergency mode, disabling SSH entirely. The fstab entry in Phase 6 uses defaults,nofail to guarantee the OS always boots and SSH remains accessible even if the data drive is absent.

Contents

- Phase 0 — OS Installation (bazzite-deck from ISO)
- Phase 1 — First Boot: SSH Bootstrap & Headless Server Configuration
- Phase 2 — Router & Network Preparation
- Phase 3 — System Package Installation (Docker · Fail2ban · SELinux tools)
- Phase 4 — Ironclad Security (SELinux Port · SSH Keys · firewalld · Fail2ban)
- Phase 5 — Docker Initialization
- Phase 6 — Data Drive Setup
- Phase 7 — Deploying Services (DuckDNS · NPM · Nextcloud AIO)
- Phase 8 — Session Management Scripts (start-kde · start-gaming · stop-session)
- Phase 9 — krdp Remote Desktop
- Phase 10 — Link & Initialize (NPM config · Nextcloud setup)
- Phase 11 — Testing & Backups
- Appendix A — Troubleshooting
- Appendix B — Normal Workflows (Quick Reference)
- Appendix C — Optional Home Assistant Integration
- Appendix D — OS Updates & Rollbacks
- Appendix E — Storage Roadmap (SnapRAID + MergerFS)
- Appendix F — Understanding Your Stack (Deep Dive)

PHASE 0 — OS Installation (bazzite-deck from ISO)

We install the bazzite-deck image directly from an ISO — no rebase from a standard Bazzite install is ever needed. This is the HTPC/living-room variant of Bazzite that ships both gamescope Gaming Mode and full KDE Plasma in a single image.

Step 1 · Download the correct ISO

Go to bazzite.gg and use the Image Picker tool. Select: Hardware = Desktop/HTPC, GPU = AMD/Intel or NVIDIA, Desktop = KDE Plasma, and enable Steam Gaming Mode (this selects the bazzite-deck variant). Download the ISO and flash it to a USB drive using Fedora Media Writer (free, cross-platform) or Balena Etcher.

> ◆ The bazzite-deck ISO is the officially correct choice for a machine that will serve as both a headless server and a gaming station. It includes gamescope-session-plus for Gaming Mode and KDE Plasma for the desktop session.

Step 2 · Install Bazzite

Boot from the USB drive and follow the Anaconda installer. Set your username, password, hostname (keep it under 20 characters), timezone, and keyboard layout. Install Bazzite to your OS drive. The installer does not require a network connection.

> ■ **WARNING:** Dual Boot Users — If you have a Windows drive in the same machine, physically disconnect it before installing Bazzite to prevent accidental data loss or bootloader modification.

Step 3 · Identify your GPU for later reference

Note which GPU type your machine has (AMD, Intel, or NVIDIA). AMD and Intel GPUs are fully supported in Gaming Mode. NVIDIA support in Gaming Mode is available but described as beta.

PHASE 1 — First Boot: SSH Bootstrap & Headless Server Configuration

After the first reboot following installation, bazzite-deck boots directly into Gaming Mode (Steam Big Picture). This is expected and intentional. Phase 1 converts the machine into a headless server during this very first boot.

> ■ **WARNING:** Do ALL steps in this phase in sequence before rebooting. The headless SDDM override in Step 4 must be in place before your next reboot. If you reboot before completing Step 4, the machine will boot to Gaming Mode again — repeat this phase.

Step 1 · Enter KDE Desktop Mode from Gaming Mode

You will see Steam Big Picture on your TV or monitor. Navigate to the Power menu (bottom-right power icon) and select Switch to Desktop. KDE Plasma will launch on the physical display.

Step 2 · Open Konsole and enable SSH

Open Konsole (the terminal emulator) from the application menu. SSH is not enabled by default on Bazzite.

```
# Enable and start the SSH daemon:
sudo systemctl enable --now sshd

# Find your server's local IP address:
ip a
# Look for inet 192.168.x.x under your ethernet interface (eth0 or enp3s0).
# Note this IP — you will need it throughout this guide.
# Example: 192.168.1.50
```

Step 3 · Disable auto-suspend (CRITICAL for servers)

Bazzite may suspend the machine after inactivity, silently killing all Docker containers. Permanently disable all suspend modes, including the suspend-then-hibernate target introduced in modern systemd.

Step A — KDE System Settings: System Settings → Power Management → Energy Saving → On AC Power → set 'Suspend session' to 'Never'. Apply.

Step B — Mask at the OS level (authoritative — do this too):

```
# Hard guarantee — works even with no desktop session active.
# suspend-then-hibernate.target is included to cover modern systemd variants.
sudo systemctl mask sleep.target suspend.target \
    hibernate.target hybrid-sleep.target suspend-then-hibernate.target

# Verify — all five must show 'masked':
systemctl status sleep.target suspend.target \
    hibernate.target hybrid-sleep.target suspend-then-hibernate.target \
    | grep -E 'Loaded:|●'
```

Step 4 · Apply the headless SDDM override (THE MOST CRITICAL STEP)

SDDM is the login screen manager. By default, bazzite-deck autologins into Gaming Mode on every boot. We create a configuration file with the zzz- prefix — which loads last — and override the autologin with empty values.

```
sudo mkdir -p /etc/sddm.conf.d

# Create the headless override file:
sudo tee /etc/sddm.conf.d/zzz-headless-server.conf > /dev/null << 'EOF'
[Autologin]
User=
Session=
Relogin=false
EOF

# Verify it was written correctly:
cat /etc/sddm.conf.d/zzz-headless-server.conf

# Apply it now without rebooting:
sudo systemctl restart sddm
# Your KDE session may end — that is expected. SSH still works.
```

◆ SDDM merges configs from /usr/lib/sddm.conf.d/ (system defaults) and /etc/sddm.conf.d/ (your overrides) in alphabetical order. Files in /etc/ take precedence, and the zzz- prefix guarantees ours is processed last. An empty User= field explicitly wins over any User= value earlier in the merge chain — this is SDDM's documented behavior.

Step 5 · Verify the headless config survives a reboot

```
# Reboot from the KDE terminal:
sudo systemctl reboot

# Wait ~60 seconds. The machine should NOT enter Gaming Mode.
# Connect from your personal computer on the temporary port 22:
ssh username@192.168.1.50

# Confirm no graphical session is running on the physical display:
loginctl list-sessions
# Expected: empty, or only your SSH session with no seat0 entry.
```

■ **WARNING:** If you cannot SSH in on port 22: boot back to Desktop Mode via Gaming Mode Power menu → Switch to Desktop, re-run Step 2, then Step 5.

PHASE 2 — Router & Network Preparation

These steps require access to your router's admin panel (typically at 192.168.1.1 or 192.168.0.1). All steps are done on your personal computer.

Step 1 · Assign a static local IP (DHCP reservation)

By MAC address, assign your server a permanent local IP in your router's DHCP reservation settings (sometimes called 'Static Lease' or 'Address Reservation'). The IP from Phase 1 Step 2 (e.g., 192.168.1.50) is what you're reserving.

Step 2 · Set up port forwarding

Port / Protocol	Purpose
80 TCP	Required by Let's Encrypt for SSL certificate issuance (HTTP challenge)
443 TCP	Your Nextcloud's public HTTPS entry point
53222 TCP	SSH — our custom hardened port (configured in Phase 4)
3478 TCP+UDP	Required ONLY if you plan to enable Nextcloud Talk later

■ **WARNING:** NEVER forward port 3389 (RDP). Port 3389 must never be forwarded or opened in firewalld. krdp is only accessible via SSH tunnel. See Phase 9.

Step 3 · Create your DuckDNS account

Go to duckdns.org and sign in with Google or GitHub. Create a free subdomain (e.g., yourname.duckdns.org) and copy your Token. Note both — you will need them in Phase 7.

PHASE 3 — System Package Installation

All packages needed for the rest of this guide are installed here in a single rpm-ostree deployment. This phase runs on port 22 (before SSH hardening). Packages must come before security hardening: the SSH hardening phase needs semanage and fail2ban — both installed here.

Step 1 · System auto-updates (no action needed)

Bazzite uses an immutable OS image: updates arrive as complete new images staged in the background, activated atomically on the next reboot. If an OS update ever causes an issue, one command returns to the previous working state:

```
rpm-ostree rollback && systemctl reboot
```

Step 2 · Add the Docker repository

```
sudo curl -fsSL https://download.docker.com/linux/fedora/docker-ce.repo \
    -o /etc/yum.repos.d/docker-ce.repo
```

Step 3 · Install all required packages (one reboot)

We check for the podman-docker shim and handle it conditionally. podman-docker provides /usr/bin/docker as a symlink to podman; if present, it conflicts with docker-ce-cli and must be removed atomically.

```
# Check whether the podman-docker compatibility shim is installed:
if rpm -q podman-docker &>/dev/null; then
    # It IS present — remove it and install Docker CE in one atomic operation:
    rpm-ostree override remove podman-docker \
        --install docker-ce --install docker-ce-cli --install containerd.io \
        --install docker-buildx-plugin --install docker-compose-plugin \
        --install fail2ban --install policycoreutils-python-utils
else
    # Not present — plain install:
    rpm-ostree install docker-ce docker-ce-cli containerd.io \
        docker-buildx-plugin docker-compose-plugin \
        fail2ban policycoreutils-python-utils
fi
systemctl reboot
```

◆ Bazzite's ujust install-docker has a confirmed bug (GitHub #2130): the Docker service fails to start after install and docker-compose is not included. The rpm-ostree method shown above is the correct path for Nextcloud AIO, which requires the native Docker socket.

PHASE 4 — Ironclad Security (SELinux Port · SSH Keys · firewalld · Fail2ban)

This phase hardens SSH access and configures the firewall. It runs after Phase 3 because it requires semanage and fail2ban — both now installed. Steps 1, 3–5 run on the server via SSH on port 22. Step 2 runs on your personal computer.

Step 1 · Label port 53222 in the SELinux policy (Server — MUST BE FIRST)

Bazzite runs SELinux in enforcing mode. Only port 22 is labelled ssh_port_t by default. If sshd is restarted on port 53222 without this label, SELinux silently blocks the bind and sshd stops listening entirely, locking you out permanently.

```
# Add port 53222 to SELinux's allowed SSH port list:
sudo semanage port -a -t ssh_port_t -p tcp 53222

# Verify — output must show '53222' in the ssh_port_t line:
sudo semanage port -l | grep ssh
# Expected: ssh_port_t tcp 53222, 22
```

■ **WARNING:** Do not skip or reorder this step. sshd will fail to bind if you restart it on port 53222 before this SELinux label is applied. Recovery requires physical console access.

Step 2 · Generate and copy SSH keys (Personal Computer)

SSH key authentication is the only login method after this phase. Passwords are disabled permanently. If you already have an SSH key pair, skip the ssh-keygen line.

Linux / macOS / WSL:

```
# Generate a modern ed25519 key pair if you don't already have one:
ssh-keygen -t ed25519

# Copy your public key to the server (still on temporary port 22):
ssh-copy-id username@192.168.1.50

# Test key login works before proceeding:
ssh username@192.168.1.50
```

Windows (PowerShell — ssh-copy-id is not available natively):

```
# Generate key (run in PowerShell):
ssh-keygen -t ed25519

# Copy public key to server (PowerShell equivalent of ssh-copy-id):
type $env:USERPROFILE\.ssh\id_ed25519.pub | ssh username@192.168.1.50
"cat >> .ssh/authorized_keys"

# Test key login:
ssh username@192.168.1.50
```

■ **WARNING:** Do not proceed past this step until key login is confirmed working. Password authentication is permanently disabled in Step 3, and you will be locked out if your key is not in place.

Step 3 · Create the SSH hardening drop-in config (Server)

We use a drop-in file in /etc/ssh/sshd_config.d/ instead of editing sshd_config directly. Drop-in files are never touched by OS updates, preventing merge conflicts from reverting your hardening.

```
sudo nano /etc/ssh/sshd_config.d/99-hardening.conf
```

Paste the entire block below, then save and exit (Ctrl+X, Y, Enter):

```
Port 53222
PubkeyAuthentication yes
PasswordAuthentication no
PermitRootLogin no
MaxAuthTries 3
LoginGraceTime 20
AllowUsers YOUR_USERNAME
```

◆ Replace YOUR_USERNAME with your actual Bazzite login name. Run whoami to confirm it.

Option	Purpose
Port 53222	Moves SSH off port 22, eliminating the vast majority of automated scan traffic.
PubkeyAuthentication yes	Explicitly enables key login — prevents silent override by other configs.
PasswordAuthentication no	Only your SSH key can log in. Passwords are fully disabled.
PermitRootLogin no	Blocks direct root access — one of the most common attack vectors.
MaxAuthTries 3 / LoginGraceTime 20	Limits the brute-force window before a connection is dropped.
AllowUsers your_name	Only your account can log in remotely. All other system users are blocked.

```
# Validate syntax — no output means no errors:
sudo sshd -t

# DO NOT restart sshd yet — open the firewall port first (Step 4).
```

Step 4 · Configure firewalld and restart sshd in the correct order

The SELinux label is in place (Step 1). Open the firewall port, restart sshd, verify it works, then remove port 22. Never skip the verification step — port 22 is your safety net if something went wrong.

```
# Step 4a — Open port 53222 in the firewall BEFORE restarting sshd:
sudo firewall-cmd --permanent --add-port=53222/tcp
sudo firewall-cmd --reload

# Step 4b — NOW restart sshd (SELinux label + firewall rule both in place):
sudo systemctl restart sshd
```

■ **WARNING:** Step 4c — TEST NOW. Open a new terminal on your personal computer: ssh -p 53222 username@192.168.1.50 — Do not proceed until this succeeds.

```
# Step 4d — Remove port 22 only after confirming port 53222 works:
sudo firewall-cmd --permanent --remove-service=ssh
sudo firewall-cmd --reload

# Verify: port 22 gone, 53222/tcp present:
sudo firewall-cmd --list-all
```

```
# Confirm SELinux port label is in effect:
sudo semanage port -l | grep ssh
# Expected: ssh_port_t tcp 53222, 22
```

◆ firewalld on Bazzite enforces rules on both IPv4 and IPv6. Docker bypasses firewalld for published ports (80, 443) — it injects nftables rules directly. Only the SSH port requires a firewalld rule.

Step 5 · Configure and enable Fail2ban

Fail2ban watches the systemd journal for failed SSH authentication attempts and bans attacking IPs via firewalld.

```
sudo nano /etc/fail2ban/jail.local
```

Paste exactly:

```
[sshd]
enabled = true
port = 53222
ignoreip = 127.0.0.1/8 192.168.0.0/16 10.0.0.0/8
backend = systemd
banaction = firewallcmd-rich-rules
```

```
sudo systemctl enable --now fail2ban

# Verify jail is active and monitoring port 53222:
sudo fail2ban-client status sshd
```

◆ backend = systemd: Fedora uses the systemd journal, not /var/log/auth.log. banaction = firewallcmd-rich-rules: Bans cover both IPv4 and IPv6. ignoreip: Exempts localhost and common private subnets (192.168.x.x, 10.x.x.x) from being banned — prevents accidental local lockout. Adjust if your LAN uses a different private range.

PHASE 5 — Docker Initialization

All remaining steps run on the server via SSH on port 53222. Docker is installed but not yet enabled or configured. This phase activates it, adds your user to the docker group, configures log rotation, and creates the shared network.

Step 1 · Enable Docker and add your user to the docker group

```
ssh -p 53222 username@192.168.1.50

# Enable Docker daemon at boot and start it now:
sudo systemctl enable --now docker

# Add your user to the docker group (allows running docker without sudo):
sudo usermod -aG docker $USER

# Log out and back in for the group change to take effect:
exit
ssh -p 53222 username@192.168.1.50
```

Step 2 · Prevent Docker logs from filling your disk

Docker logs grow indefinitely by default and will silently fill your disk over weeks. Configure log rotation before starting any containers.

```
sudo nano /etc/docker/daemon.json
```

```
{
    "log-driver": "json-file",
    "log-opts": {
        "max-size": "10m",
        "max-file": "3"
    }
}
```

```
sudo systemctl restart docker
# Each container is now capped at ~30 MB of logs total (3 files x 10 MB).
```

Step 3 · Create the shared Docker network

Nginx Proxy Manager and Nextcloud AIO communicate over a dedicated internal Docker network. Creating it first ensures both Compose stacks can reference it.

```
docker network create nextcloud-aio
# This command only needs to be run once.
```

PHASE 6 — Data Drive Setup

■ **WARNING:** Complete this before starting any Nextcloud containers. The NEXTCLOUD_DATADIR path must exist, be mounted, and have the correct SELinux label before AIO's first start. It cannot be changed afterward without a full data migration.

Step 1 · Format, mount, then label the data drive

The ordering of these commands matters critically for SELinux. We must mount the filesystem first, then apply the SELinux label. Running restorecon before mounting labels the empty mount-point directory on the OS drive — when the ext4 partition mounts over it, Nextcloud containers fail with permission denied errors.

```
# Identify your drives (find your data drive — NOT the OS drive):
lsblk

# All commands below use /dev/nvme1n1 as an example.
# Replace with your actual device name from lsblk output.

# Create a GPT partition table (WARNING: destroys all existing data):
sudo parted /dev/nvme1n1 --script mklabel gpt

# Create a single partition spanning the entire drive:
sudo parted /dev/nvme1n1 --script mkpart primary ext4 0% 100%

# Refresh the kernel's partition table view:
sudo partprobe /dev/nvme1n1

# Format the partition (note: nvme1n1p1, not nvme1n1):
sudo mkfs.ext4 /dev/nvme1n1p1

# Create the mount point:
sudo mkdir -p /srv/nextcloud-data

# Get the partition UUID for stable fstab mounting:
sudo blkid /dev/nvme1n1p1

# Add to /etc/fstab (replace UUID= value with blkid output):
sudo nano /etc/fstab
```

Add this line (note **`nofail`** — critical for headless boot resilience):

```
UUID=your-uuid   /srv/nextcloud-data   ext4   defaults,nofail   0 2
```

◆ nofail prevents systemd from dropping into emergency mode if the data drive is absent or slow to spin up. Without it, a drive failure bricks SSH access until a physical display and keyboard are attached. This is the correct and required option for any non-boot drive on a headless server.

```
# MOUNT FIRST — label must be applied to the live mounted filesystem:
sudo mount -a

# Verify the drive is mounted:
df -h /srv/nextcloud-data

# NOW apply the persistent SELinux file-context rule:
sudo semanage fcontext -a -t container_file_t "/srv/nextcloud-data(/.*)?"

# Apply the rule to the mounted filesystem:
sudo restorecon -Rv /srv/nextcloud-data
```

```
# Verify the label was applied correctly:
ls -Z /srv/nextcloud-data
# Expected: system_u:object_r:container_file_t:s0 /srv/nextcloud-data
```

◆ Why semanage fcontext instead of chcon? chcon labels are lost if the filesystem is relabelled during Fedora Atomic OS updates. semanage fcontext writes a persistent policy rule; restorecon applies it. Newly created files under /srv/nextcloud-data automatically inherit container_file_t.

PHASE 7 — Deploying Services (DuckDNS · NPM · Nextcloud AIO)

Step 1 · DuckDNS auto-updater

Keeps your domain pointed at your home IP. Uses a named volume — no bind mounts, no SELinux changes needed.

```
mkdir -p ~/docker/duckdns && cd ~/docker/duckdns
nano docker-compose.yml
```

```
services:
  duckdns:
    image: lscr.io/linuxserver/duckdns:latest
    container_name: duckdns
    restart: unless-stopped
    labels:
      - "com.centurylinklabs.watchtower.enable=true"
    environment:
      - SUBDOMAINS=yourname          # subdomain only, not the full domain
      - TOKEN=your-token-here
      - TZ=Europe/Berlin             # e.g. America/New_York, Asia/Tokyo
```

```
docker compose up -d
```

Step 2 · Nginx Proxy Manager (NPM)

Handles SSL certificates and routes external HTTPS traffic to Nextcloud. Port 81 (admin panel) is bound to 127.0.0.1 — accessible only via SSH tunnel (Phase 10).

```
mkdir -p ~/docker/npm && cd ~/docker/npm
nano docker-compose.yml
```

```
services:
  app:
    image: 'jc21/nginx-proxy-manager:latest'
    container_name: nginx-proxy-manager
    restart: unless-stopped
    labels:
      - "com.centurylinklabs.watchtower.enable=true"
    ports:
      - '80:80'              # internet access
      - '443:443'            # HTTPS
      - '127.0.0.1:81:81'    # admin panel: localhost only
    networks:
      - nextcloud-aio
    volumes:
      - ./data:/data:z               # :z = SELinux shared label
      - ./letsencrypt:/etc/letsencrypt:z
networks:
  nextcloud-aio:
    external: true        # uses the network created in Phase 5
```

```
docker compose up -d
```

◆ NPM and Nextcloud share the nextcloud-aio Docker bridge network. NPM reaches Nextcloud's Apache container by container name (nextcloud-aio-apache:11000) without going through the host network stack. Ports 80 and 443 are managed by Docker directly — never add them to firewalld.

Step 3 · Nextcloud All-in-One

Nextcloud AIO requires Docker to orchestrate multiple child containers. It connects to the Docker socket — this is why Podman is not a supported alternative.

```
mkdir -p ~/docker/nextcloud && cd ~/docker/nextcloud
nano docker-compose.yml
```

```
volumes:
  nextcloud_aio_mastercontainer:
    name: nextcloud_aio_mastercontainer

services:
  nextcloud-aio-mastercontainer:
    image: ghcr.io/nextcloud-releases/all-in-one:latest
    init: true
    restart: always
    container_name: nextcloud-aio-mastercontainer
    security_opt:
      - label:disable          # SELinux: allows Docker socket access
    labels:
      - "com.centurylinklabs.watchtower.enable=false"
    volumes:
      - nextcloud_aio_mastercontainer:/mnt/docker-aio-config
      - /var/run/docker.sock:/var/run/docker.sock:ro
    ports:
      - '127.0.0.1:8080:8080'  # setup panel: localhost only
    networks:
      - nextcloud-aio
    environment:
      - APACHE_PORT=11000
      - APACHE_IP_BINDING=127.0.0.1   # port 11000 on localhost only
      - NEXTCLOUD_DATADIR=/srv/nextcloud-data
      # Uncomment ONLY if your router lacks NAT loopback:
      # - SKIP_DOMAIN_VALIDATION=true

networks:
  nextcloud-aio:
    external: true
```

```
docker compose up -d
```

◆ Why :ro on the Docker socket? The :ro flag means the container cannot delete or replace the socket file itself. All bidirectional API communication still flows through it normally. APACHE_IP_BINDING=127.0.0.1 binds AIO's Apache port to loopback only, providing defence-in-depth against direct LAN access to port 11000 independent of Docker network rules.

PHASE 8 — Session Management Scripts

Three scripts control all graphical session transitions, called via SSH from any device. The scripts are placed in /usr/local/bin/ with root:root ownership — mandatory for any script granted passwordless sudo access.

> ◆ How the SDDM one-shot pattern works: The start scripts write a temporary autologin config, restart SDDM (which reads it and launches the session during its startup sequence), then restore the headless override in a background subshell once SDDM is confirmed active plus a 5-second stability buffer. The subshell redirects stdout/stderr to /dev/null — the SSH terminal returns immediately.

Step 1 · Understand the security model

We are granting passwordless root execution (NOPASSWD) via sudoers. Scripts must be owned by root and stored in a path the regular user cannot write to. /usr/local/bin/ is persistent and mutable on Fedora Atomic. Only root can modify files in it.

```
# Confirm your username — needed in SDDM_USER inside each script:
whoami
```

Step 2 · Write start-kde.sh

Replace yourusername with your actual Bazzite login name in SDDM_USER.

```
sudo nano /usr/local/bin/start-kde.sh
```

Paste the following as the complete file content, then save and exit nano (Ctrl+X, Y, Enter):

```
#!/bin/bash
# start-kde.sh — Launch a KDE Wayland session on the physical display.
# Refuses if a graphical session is already active on seat0.
# Location: /usr/local/bin/start-kde.sh (root-owned, not user-writable)
# Usage: ssh -p 53222 user@host 'sudo start-kde.sh'

SDDM_USER="yourusername"    # ← REPLACE with your Bazzite username
KDE_SESSION="plasma"
LAUNCH_CONF="/etc/sddm.conf.d/zz-session-launch.conf"
HEADLESS_CONF="/etc/sddm.conf.d/zzz-headless-server.conf"

log() { echo "[start-kde $(date '+%H:%M:%S')] $*"; }

ACTIVE=$(loginctl list-sessions --no-legend 2>/dev/null \
    | awk '{print $1}' \
    | while read -r sid; do
        seat=$(loginctl show-session "$sid" -p Seat --value 2>/dev/null) ||
continue
        state=$(loginctl show-session "$sid" -p State --value 2>/dev/null) ||
continue
        if [[ "$seat" == "seat0" ]] && \
           [[ "$state" == "active" || "$state" == "online" ]]; then
            echo "$sid"; break
        fi
    done)

if [[ -n "$ACTIVE" ]]; then
    log "ERROR: A graphical session ($ACTIVE) is already active on seat0."
    log "Log out of the current session first, then re-run this script."
    exit 1
fi
```

```
rm -f "$HEADLESS_CONF"
mkdir -p /etc/sddm.conf.d
cat > "$LAUNCH_CONF" << EOF
[Autologin]
User=${SDDM_USER}
Session=${KDE_SESSION}
Relogin=false
EOF

log "SDDM autologin config written for KDE Wayland."
log "Restarting SDDM..."
systemctl reset-failed sddm
systemctl restart sddm

# Restore headless config in background.
# systemctl restart sddm is synchronous — SDDM has read the config by the time
# we reach here. We wait for SDDM to report active (confirming the restart
# cycle completed), then add a 5-second buffer for session initialisation.
# stdout/stderr redirected to /dev/null — SSH returns immediately.
(
    for i in {1..20}; do
        systemctl is-active --quiet sddm && break
        sleep 1
    done
    sleep 5
    rm -f "$LAUNCH_CONF"
    cat > "$HEADLESS_CONF" << 'EOF'
[Autologin]
User=
Session=
Relogin=false
EOF
) >/dev/null 2>&1 &
disown $!

log "KDE Wayland session launching on physical display."
```

```
sudo chown root:root /usr/local/bin/start-kde.sh
sudo chmod 755 /usr/local/bin/start-kde.sh
```

Step 3 · Write start-gaming.sh

Same SDDM pattern, but for the gamescope Gaming Mode session. Replace yourusername.

```
sudo nano /usr/local/bin/start-gaming.sh
```

Paste the following as the complete file content, then save and exit:

```
#!/bin/bash
# start-gaming.sh — Launch Gaming Mode on the physical display.
# REFUSES if any graphical session is active on seat0.
# Location: /usr/local/bin/start-gaming.sh (root-owned, not user-writable)
# Usage: ssh -p 53222 user@host 'sudo start-gaming.sh'

SDDM_USER="yourusername"    # ← REPLACE with your Bazzite username
GAMING_SESSION="gamescope-session-plus"
LAUNCH_CONF="/etc/sddm.conf.d/zz-session-launch.conf"
HEADLESS_CONF="/etc/sddm.conf.d/zzz-headless-server.conf"

log() { echo "[start-gaming $(date '+%H:%M:%S')] $*"; }
```

```
ACTIVE=$(loginctl list-sessions --no-legend 2>/dev/null \
    | awk '{print $1}' \
    | while read -r sid; do
        seat=$(loginctl show-session "$sid" -p Seat --value 2>/dev/null) ||
continue
        state=$(loginctl show-session "$sid" -p State --value 2>/dev/null) ||
continue
        if [[ "$seat" == "seat0" ]] && \
           [[ "$state" == "active" || "$state" == "online" ]]; then
            echo "$sid"; break
        fi
    done)

if [[ -n "$ACTIVE" ]]; then
    log "ERROR: A session ($ACTIVE) is already active on seat0."
    log "Log out using the session's own logout button, then re-run this script."
    exit 1
fi

SESSION_FILE="/usr/share/wayland-sessions/${GAMING_SESSION}.desktop"
if [[ ! -f "$SESSION_FILE" ]]; then
    log "ERROR: Session file not found: $SESSION_FILE"
    log "Ensure bazzite-deck was installed (not the standard bazzite image)."
    exit 1
fi

rm -f "$HEADLESS_CONF"
mkdir -p /etc/sddm.conf.d
cat > "$LAUNCH_CONF" << EOF
[Autologin]
User=${SDDM_USER}
Session=${GAMING_SESSION}
Relogin=false
EOF

log "SDDM autologin config written for Gaming Mode."
log "Restarting SDDM..."
systemctl reset-failed sddm
systemctl restart sddm

# Same deterministic restore pattern as start-kde.sh.
(
    for i in {1..20}; do
        systemctl is-active --quiet sddm && break
        sleep 1
    done
    sleep 5
    rm -f "$LAUNCH_CONF"
    cat > "$HEADLESS_CONF" << 'EOF'
[Autologin]
User=
Session=
Relogin=false
EOF
) >/dev/null 2>&1 &
disown $!

log "Gaming Mode launching on physical display."
```

```
sudo chown root:root /usr/local/bin/start-gaming.sh
sudo chmod 755 /usr/local/bin/start-gaming.sh
```

Step 4 · Write stop-session.sh

Emergency fallback — terminates whichever graphical session is active on seat0. For normal use, log out via KDE's own logout button or the Power menu in Steam Big Picture.

```
sudo nano /usr/local/bin/stop-session.sh
```

Paste the following as the complete file content, then save and exit:

```
#!/bin/bash
# stop-session.sh — Emergency: terminate whichever seat0 session is active.
# Normal use: log out via KDE's logout button or Steam's Power menu.
# Location: /usr/local/bin/stop-session.sh (root-owned, not user-writable)
# Usage: ssh -p 53222 user@host 'sudo stop-session.sh'

LAUNCH_CONF="/etc/sddm.conf.d/zz-session-launch.conf"
HEADLESS_CONF="/etc/sddm.conf.d/zzz-headless-server.conf"

log() { echo "[stop-session $(date '+%H:%M:%S')] $*"; }

# Always restore headless config first
rm -f "$LAUNCH_CONF"
cat > "$HEADLESS_CONF" << 'EOF'
[Autologin]
User=
Session=
Relogin=false
EOF

log "SDDM configs confirmed in headless state."

ACTIVE=$(loginctl list-sessions --no-legend 2>/dev/null \
    | awk '{print $1}' \
    | while read -r sid; do
        seat=$(loginctl show-session "$sid" -p Seat --value 2>/dev/null) ||
continue
        state=$(loginctl show-session "$sid" -p State --value 2>/dev/null) ||
continue
        if [[ "$seat" == "seat0" ]] && \
           [[ "$state" == "active" || "$state" == "online" ]]; then
            echo "$sid"; break
        fi
    done)

if [[ -n "$ACTIVE" ]]; then
    log "Terminating session $ACTIVE on seat0..."
    loginctl terminate-session "$ACTIVE"
    log "Done. SDDM login screen should appear on the physical display."
else
    log "No active seat0 session found. Nothing to terminate."
fi

log "WARNING: All unsaved work in the terminated session is lost."
```

```
sudo chown root:root /usr/local/bin/stop-session.sh
sudo chmod 755 /usr/local/bin/stop-session.sh
```

Step 5 · Configure targeted sudo for the scripts

```
# Open a new sudoers drop-in file using the safe visudo editor:
sudo visudo -f /etc/sudoers.d/session-scripts
```

```
# Paste these three lines (replace yourusername with 'whoami' output):
yourusername ALL=(ALL) NOPASSWD: /usr/local/bin/start-kde.sh
yourusername ALL=(ALL) NOPASSWD: /usr/local/bin/start-gaming.sh
yourusername ALL=(ALL) NOPASSWD: /usr/local/bin/stop-session.sh
```

```
# Save and exit visudo. Validate syntax:
sudo visudo -c -f /etc/sudoers.d/session-scripts
# Expected: /etc/sudoers.d/session-scripts: parsed OK
```

Step 6 · Test the scripts manually

```
# Launch KDE Wayland on the physical display:
sudo start-kde.sh
# Expected: KDE appears. SSH terminal returns promptly.

# While KDE is running, try to launch Gaming Mode (must be refused):
sudo start-gaming.sh
# Expected: ERROR — session already active.

# Log out of KDE via its own logout button. Then:
sudo start-gaming.sh
# Expected: Gaming Mode appears on the physical display.

# Test emergency stop:
sudo stop-session.sh
# Expected: Gaming Mode terminates. SDDM login screen appears.

# Confirm session files exist:
ls /usr/share/wayland-sessions/
# Must include: gamescope-session-plus.desktop and plasma.desktop

# Confirm scripts are root-owned:
ls -la /usr/local/bin/start-kde.sh \
        /usr/local/bin/start-gaming.sh \
        /usr/local/bin/stop-session.sh
# Expected: -rwxr-xr-x 1 root root ... for all three files
```

PHASE 9 — krdp Remote Desktop

krdp is KDE's native Wayland RDP server, integrated into KDE Plasma 6.1+. It shares the running KDE Wayland compositor session — no virtual framebuffer, no X11, no extra packages required. Monitors can be off.

> ■ **WARNING:** SECURITY: krdp listens on 0.0.0.0 (all interfaces), not just localhost. Port 3389 is reachable on your LAN IP unless blocked. firewalld blocks it because we never add port 3389 to any allowed rule. NEVER open port 3389 in firewalld. Access is exclusively via SSH tunnel.

Step 1 · Enable krdp in KDE System Settings

Requires a running KDE session. Run sudo start-kde.sh first if needed.

Step	Action
1. Open System Settings	Navigate to: System Settings → Remote Desktop
2. Enable RDP	Toggle 'Enable Remote Desktop' to ON
3. Set credentials	Set a username and password for RDP login (can differ from system login)
4. Check the port	Default port is 3389. Leave it as-is.
5. Apply	Click Apply. krdp begins listening immediately on all interfaces.

Step 2 · Verify firewall protection (mandatory after enabling krdp)

```
# Confirm krdp is listening:
ss -tlnp | grep 3389
# Expected: port 3389 is listening on all interfaces — this is normal.

# Verify firewalld is NOT allowing port 3389:
sudo firewall-cmd --list-all
# Search for '3389' — it must NOT appear in 'ports:' or 'services:'.

# If 3389 appears, remove it immediately:
# sudo firewall-cmd --permanent --remove-port=3389/tcp
# sudo firewall-cmd --reload

# Final test: try connecting from another LAN device to
# 192.168.1.50:3389 WITHOUT an SSH tunnel — it must be refused.
```

Step 3 · Connect via SSH tunnel from your personal computer

```
# On your PERSONAL COMPUTER — keep this terminal open:
ssh -N -L 3389:127.0.0.1:3389 -p 53222 username@your-server-ip

# Then connect any RDP client to: localhost:3389
# Use the krdp credentials set in System Settings (Step 1).
```

Client	Platform	Connect to
Remote Desktop Connection (built-in)	Windows	localhost:3389
Microsoft Remote Desktop	macOS	localhost:3389

Remmina	Linux	localhost:3389
Microsoft Remote Desktop	Android	Run SSH tunnel in Termux first

◆ krdp only runs while a KDE Wayland session is active. When KDE is not running (e.g., during Gaming Mode), krdp is not running and RDP connections will be refused. Keep the SSH tunnel terminal open — closing it disconnects the RDP session immediately.

PHASE 10 — Link & Initialize Nextcloud

Step 1 · Access admin panels via SSH tunnel

The admin panels are bound to 127.0.0.1 — invisible to your local network. Create an SSH tunnel from your personal computer and leave it running while you use the panel.

```
# NPM admin panel → http://localhost:8081
ssh -N -L 8081:127.0.0.1:81 -p 53222 username@192.168.1.50

# Nextcloud AIO setup panel → https://localhost:9443
ssh -N -L 9443:127.0.0.1:8080 -p 53222 username@192.168.1.50

# Run each tunnel in a separate terminal window and leave it open
# while you use the corresponding panel.
```

◆ -N means 'do not execute a remote command' (tunnel only). -L binds a local port to a remote address through the encrypted SSH connection.

Step 2 · Configure Nginx Proxy Manager

Start the NPM tunnel, open http://localhost:8081. Log in with admin@example.com / changeme — change both immediately. Go to Hosts → Proxy Hosts → Add Proxy Host:

Field	Value
Domain Names	yourname.duckdns.org
Scheme	http
Forward Hostname / IP	nextcloud-aio-apache (container name — not an IP address)
Forward Port	11000
Toggles	Cache Assets · Block Common Exploits · Websockets Support — all ON
SSL tab	Request new SSL Certificate · Force SSL · HTTP/2 Support · Agree to ToS → Save

Step 3 · Fix local network access (NAT loopback)

Many routers cannot route traffic from your local network back through their own public IP. yourname.duckdns.org resolves to your public IP, but the connection fails from inside your home network.

Solution 1 — Router local DNS override (recommended): Add yourname.duckdns.org → 192.168.1.50 in your router's 'Local DNS', 'DNS Entries', or 'Static Hosts' settings.

Solution 2 — Pi-hole local DNS: Add the same DNS override in Pi-hole's 'Local DNS Records'.

Solution 3 — SKIP_DOMAIN_VALIDATION (last resort): Uncomment the line in the Nextcloud AIO compose file, then cd ~/docker/nextcloud && docker compose down && docker compose up -d.

■ **WARNING:** Client /etc/hosts files do NOT solve this. Adding yourname.duckdns.org to /etc/hosts on your PC only helps your browser. The domain validation request originates from inside the Docker container on the server — not from your client device.

Step 4 · Initialize Nextcloud AIO

Stop the NPM tunnel (Ctrl+C). Start the Nextcloud AIO tunnel. Go to https://localhost:9443 and accept the browser security warning (expected — this is AIO's self-signed certificate). Copy the setup password. Enter your domain: yourname.duckdns.org. Choose optional containers and click Start Containers. Wait 5–10 minutes.

PHASE 11 — Testing & Backups

DNS Test

```
# Bypasses your local DNS override and queries Google DNS directly.
# The result must be your home's public IP.
nslookup yourname.duckdns.org 8.8.8.8
```

External Test

Turn off Wi-Fi on your phone and visit https://yourname.duckdns.org over cellular. A green padlock and the Nextcloud login screen confirms end-to-end functionality.

Session Management Test

```
ssh -p 53222 username@your-server-ip 'sudo start-kde.sh'
# KDE appears on the physical display. SSH terminal returns promptly.

ssh -p 53222 username@your-server-ip 'sudo start-gaming.sh'
# Must output ERROR — KDE is still active.

# Log out of KDE via its logout button, then:
ssh -p 53222 username@your-server-ip 'sudo start-gaming.sh'
# Gaming Mode appears on the physical display.

ssh -p 53222 username@your-server-ip 'sudo stop-session.sh'
# Gaming Mode terminates. SDDM login screen appears.
```

System State Validation Checklist

Run these checks after completing all phases. Each has a specific expected output — if any differ, the corresponding phase has a problem.

```
# 1. Headless SDDM default:
ls /etc/sddm.conf.d/
# Expected: zzz-headless-server.conf (only this file at idle)

# 2. No active graphical session at idle:
loginctl list-sessions
# Expected: empty or only your SSH session, no seat0 entry

# 3. SELinux port label:
sudo semanage port -l | grep ssh
# Expected: ssh_port_t tcp 53222, 22

# 4. Nextcloud data drive SELinux label:
ls -Z /srv/nextcloud-data
# Expected: system_u:object_r:container_file_t:s0

# 5. Docker running without sudo:
docker info 2>&1 | grep 'Server Version'
# Expected: Server Version: (no permission denied error)

# 6. Fail2ban active on port 53222:
sudo fail2ban-client status sshd
# Expected: 'Status for the jail: sshd' with 'Port list: 53222'
```

```
# 7. Scripts owned by root:
ls -la /usr/local/bin/start-kde.sh \
        /usr/local/bin/start-gaming.sh \
        /usr/local/bin/stop-session.sh
# Expected: -rwxr-xr-x 1 root root ... for all three files

# 8. All containers healthy:
docker ps --format 'table {{.Names}}\t{{.Status}}'
# Expected: duckdns, nginx-proxy-manager, nextcloud-aio-mastercontainer all 'Up'

# 9. Port security — confirm only 53222/tcp is open, never 22, 80, 443, or 3389:
sudo firewall-cmd --list-all
# Expected: ports: 53222/tcp — all others strictly absent

# 10. fstab nofail — headless boot resilience:
grep nextcloud-data /etc/fstab
# Expected: contains nofail in the options field

# 11. All suspend targets masked:
systemctl status sleep.target suspend.target hibernate.target \
    hybrid-sleep.target suspend-then-hibernate.target | grep -E 'Loaded:|●'
# Expected: all five show masked
```

Backups

Inside the AIO setup panel (https://localhost:9443) you will find the built-in Borg Backup tool. Configure it before storing any personal data in Nextcloud.

Set a destination path (e.g., /mnt/backups or a mounted external USB drive). Set backups to run daily. Perform a test restore immediately after the first backup completes.

> ■ **WARNING:** Perform a test restore once a month. A backup you have never tested cannot be trusted.

APPENDIX A — Troubleshooting

Machine boots to Gaming Mode after initial install (expected once)

```
# SSH is not available yet.
# Go to Gaming Mode → Power menu → Switch to Desktop.
# Enable SSH and apply the headless SDDM override (Phase 1 Steps 2-4).
```

start-kde.sh or start-gaming.sh: 'session already active' but nothing visible

A stale logind session may be present. List sessions, note the session ID, then terminate it by its ID:

```
loginctl list-sessions
# Note the session ID from the output (e.g. "3" or "c1").
loginctl terminate-session SESSION_ID    # replace with actual ID
sleep 2
sudo start-kde.sh       # or: sudo start-gaming.sh
```

SDDM shows login screen instead of autologining

```
# Check the exact session name (must match .desktop filename without extension):
ls /usr/share/wayland-sessions/
# KDE: plasma.desktop → Session=plasma
# Gaming: gamescope-session-plus.desktop → Session=gamescope-session-plus

grep 'KDE_SESSION=' /usr/local/bin/start-kde.sh
grep 'GAMING_SESSION=' /usr/local/bin/start-gaming.sh

# Check SDDM logs:
journalctl -u sddm -n 50 --no-pager
```

krdp RDP connection refused

```
# 1. Confirm KDE is running:
loginctl list-sessions    # must show a seat0 session

# 2. Confirm krdp is listening:
ss -tlnp | grep 3389    # must show port 3389 listening

# 3. If nothing: re-check Remote Desktop is enabled in KDE System Settings.

# 4. Confirm the SSH tunnel is open on your personal computer:
#    The 'ssh -N -L 3389:127.0.0.1:3389' terminal must still be running.
```

Docker containers not running after a reboot

```
sudo systemctl status docker
# If not enabled:
sudo systemctl enable --now docker
cd ~/docker/npm && docker compose up -d
cd ~/docker/nextcloud && docker compose up -d
cd ~/docker/duckdns && docker compose up -d
docker ps --format 'table {{.Names}}\t{{.Status}}'
```

zz-session-launch.conf persists after a script run (power loss edge case)

The background cleanup was interrupted. Manual fix:

```
sudo rm -f /etc/sddm.conf.d/zz-session-launch.conf
sudo tee /etc/sddm.conf.d/zzz-headless-server.conf > /dev/null << 'EOF'
[Autologin]
User=
Session=
Relogin=false
EOF

ls /etc/sddm.conf.d/
# Expected: only zzz-headless-server.conf
```

Data drive not mounting — system dropped to emergency mode

This should not happen with nofail in fstab (Phase 6). If emergency mode occurs on an older setup without nofail:

```
# Attach a display and keyboard.
# At the emergency prompt, edit fstab:
nano /etc/fstab
# Find the /srv/nextcloud-data line and add nofail to the options column.
# Then reboot:
systemctl reboot
```

APPENDIX B — Normal Workflows (Quick Reference)

I want to: Use KDE on the TV or monitor

```
ssh -p 53222 username@your-server-ip 'sudo start-kde.sh'
# KDE Wayland session appears.
# To stop: Application Menu → Leave → Log Out.
```

I want to: Work remotely via RDP

```
# Step 1 — Start KDE:
ssh -p 53222 username@your-server-ip 'sudo start-kde.sh'

# Step 2 — Open the SSH tunnel (keep this terminal open):
ssh -N -L 3389:127.0.0.1:3389 -p 53222 username@your-server-ip

# Step 3 — Connect your RDP client to: localhost:3389
```

I want to: Game

```
# If KDE is running: log out first via KDE's logout button. Then:
ssh -p 53222 username@your-server-ip 'sudo start-gaming.sh'
# To stop: Power menu in Steam Big Picture → Exit to Desktop.
```

I want to: Force-stop a stuck session

```
ssh -p 53222 username@your-server-ip 'sudo stop-session.sh'
# WARNING: All unsaved work in the terminated session is lost.
```

I want to: Verify what is currently running

```
loginctl list-sessions
# seat0 entry = physical display session active

loginctl show-session -p Type -p Desktop
# Type=wayland, Desktop=KDE → KDE session
# Type=wayland, Desktop=gamescope → Gaming Mode

ls /etc/sddm.conf.d/
# Healthy: only zzz-headless-server.conf present
```

APPENDIX C — Optional Home Assistant Integration

Because the scripts are plain SSH commands, adding Home Assistant control requires only a few lines in configuration.yaml — zero changes to the scripts themselves.

Step 1 · Generate a dedicated SSH keypair for Home Assistant

```
# Create the target directory first:
mkdir -p ~/docker/ai-home/homeassistant-data/.ssh

# Generate a dedicated key for HA:
ssh-keygen -t ed25519 \
    -f ~/docker/ai-home/homeassistant-data/.ssh/id_ed25519_ha -N ''

# Authorise this key on the server:
cat ~/docker/ai-home/homeassistant-data/.ssh/id_ed25519_ha.pub \
    >> ~/.ssh/authorized_keys

# Pre-populate known_hosts so HA does not fail on first connection:
ssh-keyscan -p 53222 127.0.0.1 \
    >> ~/docker/ai-home/homeassistant-data/.ssh/known_hosts

# Lock down permissions:
chmod 700 ~/docker/ai-home/homeassistant-data/.ssh
chmod 600 ~/docker/ai-home/homeassistant-data/.ssh/id_ed25519_ha
chmod 644 ~/docker/ai-home/homeassistant-data/.ssh/id_ed25519_ha.pub
chmod 644 ~/docker/ai-home/homeassistant-data/.ssh/known_hosts
```

◆ The key lives inside the HA data directory (homeassistant-data/.ssh/), accessible inside the container at /config/.ssh/. No additional volume mount needed. The key persists through container updates.

Step 2 · Add shell commands to configuration.yaml

```
# In ~/docker/ai-home/homeassistant-data/configuration.yaml:
shell_command:
  start_kde: >-
    ssh -i /config/.ssh/id_ed25519_ha
    -o StrictHostKeyChecking=yes
    -o UserKnownHostsFile=/config/.ssh/known_hosts
    -p 53222 yourusername@127.0.0.1
    'sudo start-kde.sh'
  start_gaming: >-
    ssh -i /config/.ssh/id_ed25519_ha
    -o StrictHostKeyChecking=yes
    -o UserKnownHostsFile=/config/.ssh/known_hosts
    -p 53222 yourusername@127.0.0.1
    'sudo start-gaming.sh'
  stop_session: >-
    ssh -i /config/.ssh/id_ed25519_ha
    -o StrictHostKeyChecking=yes
    -o UserKnownHostsFile=/config/.ssh/known_hosts
    -p 53222 yourusername@127.0.0.1
    'sudo stop-session.sh'
```

```
docker restart homeassistant
# Then create buttons in your HA dashboard calling each shell_command.
```

◆ Why 127.0.0.1? Home Assistant uses network_mode: host. Port 53222 on the host is reachable

> from inside the HA container as 127.0.0.1:53222. Why UserKnownHostsFile? The HA process runs as root inside the container. Without an explicit known_hosts path, SSH looks in /root/.ssh/known_hosts, which does not exist.

APPENDIX D — OS Updates & Rollbacks

Bazzite stages OS updates silently in the background. Applying them requires a reboot — during which your server is offline for 1–2 minutes. All containers come back up automatically (restart: always / restart: unless-stopped).

```
# If an update causes any problem:
rpm-ostree rollback && systemctl reboot
```

```
# List available stable builds:
skopeo list-tags docker://ghcr.io/ublue-os/bazzite-deck \
    | grep -- 'stable-' | sort -rV

# Rebase to a specific build:
rpm-ostree rebase ostree-image-signed:docker://ghcr.io/ublue-os/bazzite-
deck:stable-YEARMONTHDAY
```

Watchtower — Auto-update containers

Docker containers do not update themselves. Watchtower automates this using opt-in mode: only containers marked with enable=true (DuckDNS and NPM). Nextcloud AIO containers are excluded and updated manually via the AIO panel.

```
mkdir -p ~/docker/watchtower && cd ~/docker/watchtower
nano docker-compose.yml
```

```
services:
  watchtower:
    image: containrrr/watchtower
    restart: unless-stopped
    security_opt:
      - label:disable
    environment:
      - TZ=Europe/Berlin
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
    command: --label-enable --cleanup --schedule "0 0 4 * * *"
    # Only updates containers with enable=true (DuckDNS, NPM).
    # All Nextcloud AIO containers are untouched. Runs at 04:00.
```

```
docker compose up -d
```

APPENDIX E — Storage Roadmap (SnapRAID + MergerFS)

This appendix describes the storage expansion path when you add additional HDDs. It does not apply to initial setup. SnapRAID + MergerFS works with drives of different sizes, has no continuous write overhead (parity computed once daily), and files remain independently accessible on individual drives.

```
# 1. Add the mergerfs Copr repository:
sudo curl -fsSL \
    "https://copr.fedorainfracloud.org/coprs/errornointernet/mergerfs/repo/fedora-
$(rpm -E %fedora)/errornointernet-mergerfs-fedora-$(rpm -E %fedora).repo" \
    -o /etc/yum.repos.d/mergerfs.repo

# 2. Install MergerFS + SnapRAID:
rpm-ostree install mergerfs snapraid
systemctl reboot

# 3. Format and configure HDDs (parted + mkfs.ext4 as in Phase 6)

# 4. Set up MergerFS unified pool: /mnt/disk1 + /mnt/disk2 → /srv/media

# 5. Configure SnapRAID with one disk as parity

# 6. Migrate Nextcloud data to the new pool:
cd ~/docker/nextcloud && docker compose stop
sudo rsync -aHAX /srv/nextcloud-data/ /srv/media/nextcloud/

# 7. Update NEXTCLOUD_DATADIR in compose file

# 8. Apply SELinux label to new path:
sudo semanage fcontext -a -t container_file_t "/srv/media/nextcloud(/.*)?"
sudo restorecon -Rv /srv/media/nextcloud

# 9. Start Nextcloud AIO:
docker compose up -d
```

APPENDIX F — Understanding Your Stack (Deep Dive)

Why does stop-session.sh not kill the SSH session?

SSH runs as a systemd service and SSH connections are logind sessions on a virtual terminal (TTY) with no assigned seat. loginctl terminate-session targets only the seat0 session — the one owning the physical display, keyboard, and mouse. Your SSH session has no seat assignment and is completely unaffected.

Why can't Gaming Mode and KDE run simultaneously?

Both are Wayland compositors competing for the same physical GPU and display output (seat0). A Wayland compositor takes exclusive ownership of the GPU's DRM device via the DRM master mechanism. Only one process can hold DRM master on a given seat at a time. gamescope additionally uses direct scanout which requires exclusive GPU access. This is a fundamental hardware constraint.

Why does krdp work with monitors off?

krdp captures frames directly from the KDE Wayland compositor via PipeWire's screen capture protocol. The compositor renders frames to an internal framebuffer regardless of whether a physical display is connected or in DPMS sleep. Monitor power state is irrelevant to compositor rendering.

Why does logging out of KDE land at SDDM login, not Gaming Mode?

When KDE logs out, it hands control back to SDDM. Since the headless override (zzz-headless-server.conf) was restored by the background subshell, SDDM finds User= empty and shows a login screen. The server is in headless idle — this is correct and intentional.

Why does the SSH terminal return immediately after the start scripts?

The background subshell has stdout and stderr redirected to /dev/null before being detached with disown. The SSH daemon closes the connection once all processes that inherited the original stdout/stderr have exited. The subshell holds no reference to the SSH session's file descriptors, so the connection closes as soon as the main script body finishes — instantly.

Why the deterministic subshell pattern instead of a fixed sleep?

The original sleep 7 is sufficient on idle systems because systemctl restart ssdm is synchronous — SDDM has already read the autologin config by the time restart returns. However, on systems under heavy load (Borg backup, Nextcloud cron), systemd may report the service as active before all startup tasks complete. The updated pattern (poll for SDDM active, then 5-second buffer) is more robust on loaded systems without sacrificing correctness on idle ones.

Why bazzite-deck and not standard bazzite + rebase?

Starting with bazzite-deck directly: (1) eliminates an entire rebase phase and Docker reinstall, (2) ensures gamescope-session-plus is part of the signed base image, (3) means every OS update includes Gaming Mode infrastructure, (4) keeps the system on the signed image track from day one.

Why is Docker installed via rpm-ostree rather than Podman?

Nextcloud AIO manages multiple child containers by connecting to the Docker socket. Its mastercontainer requires a native Docker API for this orchestration. Rootless Podman's Docker API compatibility is insufficient for AIO's needs. rpm-ostree layering is the correct method for system-level packages on Fedora Atomic.

Why are the scripts in /usr/local/bin/ and owned by root?

The scripts have NOPASSWD sudo access. If the script file were writable by the regular user (as in ~/scripts/), any process running as that user could overwrite it with arbitrary commands and invoke sudo to execute them as root — a complete privilege escalation. /usr/local/bin/ is persistent and mutable on Fedora Atomic; only root can modify files in it with root:root ownership and 755 permissions.

Why must SELinux be updated before restarting sshd on port 53222?

The ssh_port_t type labels ports sshd is permitted to bind to. Port 22 carries this label by default. Port 53222 does not. When sshd attempts to bind(2) to port 53222, the SELinux kernel module denies the syscall because port 53222 lacks ssh_port_t. sshd starts but does not listen on any port. The remedy is semanage port -a -t ssh_port_t -p tcp 53222, which writes a persistent entry into the SELinux port context database.

Why must the filesystem be mounted before running restorecon?

SELinux file contexts are stored as extended attributes (xattrs) on filesystem inodes. Running restorecon on /srv/nextcloud-data before mounting labels the OS drive's inode. When the ext4 partition mounts, the mount operation replaces that inode with the filesystem's own root inode, which defaults to unlabeled_t on a freshly formatted partition. Nextcloud containers in the container_t domain cannot write to unlabeled_t paths.

Why does krdp listen on all interfaces?

krdp uses QHostAddress::Any, which binds to all network interfaces. There is currently no configuration option to restrict this to localhost only (KDE bug #515737). firewalld compensates by dropping all inbound connections to port 3389 from the LAN. The SSH tunnel provides the only path to reach krdp from outside the server.

Why does nofail matter for headless servers?

Without nofail, systemd treats a failed mount as a fatal boot error and drops the system into emergency mode — an interactive recovery shell that requires a physical display and keyboard. On a headless server, this is equivalent to a brick: SSH is not started and the machine is unreachable remotely. With nofail, systemd logs the mount failure and continues booting, preserving SSH access and allowing remote investigation and repair.

Time synchronization

Incorrect system time silently breaks TLS certificate validation and Nextcloud login sessions. Bazzite uses chronyd — never enable systemd-timesyncd on Bazzite; it conflicts with chronyd.

```
timedatectl
# Confirm: System clock synchronized: yes

systemctl status chronyd
# If not running: sudo systemctl enable --now chronyd
```

Disk usage monitoring

```
df -h
# When any partition hits 90%+ usage, act immediately.
# Clean old Docker images: docker system prune
```

This guide is fully self-contained. All phases are additive — they do not conflict with one another. All Docker containers, volumes, network configuration, and firewall rules operate independently of graphical sessions and SDDM restarts.